

`flock()` not only locks an existing document, but also prevents the file from further writing or reading. The `flock()` function always requires a file pointer. Therefore, sometimes you have to use one special lock file to prevent file access. A file locking system requires a modern approach. In PHP, the flock system is used virtually at different stages.

In PHP, a complete file can be locked in an advisory pattern. This suggests that all other programs are entitled to use the similar locking manner unless they do not work. A file locked using flock can be released by `fclose()`.

flock considers the file handle as the first parameter, while the entire lock operation follows next. The operations applied in PHP include LOCK_SH (requests a shared lock), LOCK_EX (requests an exclusive lock), and LOCK_UN (releases a lock). In a PHP file, flock can be used in the following manner:

```
<?php
$file_ptr = fopen("XYZ.txt", "w");
if (flock($file_ptr, LOCK_EX)) {
    print "Got lock!\n";
    sleep(10);
    flock($file_ptr, LOCK_UN);
}
?>
```

File locking in PHP requires a complete unconventional approach. It cannot be done in the original Microsoft's FAT file system's version used on Windows 95 and 98. Both NTFS and FAT32 can work for file locking in PHP. All the processes themselves get locked in PHP by default. The example below can focus on the concept:

```
<?php
$file_ptr = fopen("XYZ.txt", "w");
if (flock($file_ptr, LOCK_EX)) {
    print "Got lock!\n";
    sleep(10);
    flock($file_ptr, LOCK_UN);
}
?>
```